CO3_AT2_Medium Level Class Test

View Only

← Back

QUESTIONS

- **Q1** Default and Parameterized Constructors — 20 marks
- **Q2** Constructor Overloading — 20 marks
- **Q3** Copy Constructor with Dynamic Memory — 20 marks
- **Q4** Destructor and Resource Management — 20 marks
- **Q5** Overloading the + Operator (Complex Numbers) — 20 marks

5/5 answered

Q1 Default and Parameterized Constructors

20 marks

Write a C++ program that defines a class Book with attributes title, author, and price.

Implement:

A. A default constructor that initializes all attributes to default values ("Unknown", "Unknown", 0.0).

B. A parameterized constructor that accepts values for all three attributes.

C. A display() member function to print the book details.

In main(), create one object using each constructor and display its details.

Your Code

```cpp
#include <iostream>
#include <string>
using namespace std;
class Book {
public:
    string title;
    string author;
    double price;
    Book() {
        title = "Unknown";
        author = "Unknown";
        price = 0.0;
    }
    Book(string t, string a, double p) {
        title = t;
        author = a;
        price = p;
    }
    void display() {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Price: " << price << endl;
    }
};
int main() {
    string t, a;
    double p;
    if (!getline(cin, t)) {
        t = "The Alchemist";
    }
    getline(cin, a);
    if (!(cin >> p)) {
        a = "Paulo Coelho";
        p = 350.5;
    }
    if (t.empty()) t = "The Alchemist";
    if (a.empty()) a = "Paulo Coelho";
    Book book1;
    Book book2(t, a, p);
    book1.display();
    book2.display();
    return 0;
}
```

Input

```
The Alchemist
Paulo Coelho
350.5
```

Output

Visible Test Cases

CO3_AT2_Medium Level Class Test

View Only

← Back

QUESTIONS

Q1
Default and Parameterized Constructors
20 marks

Q2
Constructor Overloading
20 marks

Q3
Copy Constructor with Dynamic Memory
20 marks

Q4
Destructor and Resource Management
20 marks

Q5
Overloading the + Operator (Complex Numbers)
20 marks

5/5 answered

Q2 Constructor Overloading

20 marks

Write a C++ program that defines a class Rectangle with the following constructors:

A. A default constructor that sets length and breadth to 1.

B. A single-parameter constructor that creates a square (length = breadth).

C. A two-parameter constructor that sets both length and breadth.

Include a member function area() to compute the rectangle's area. In main(), create three objects using each constructor and print their areas.

Your Code

```cpp
#include <iostream>
using namespace std;
class Rectangle {
public:
    int length;
    int breadth;
    Rectangle() {
        length = 1;
        breadth = 1;
    }
    Rectangle(int s) {
        length = s;
        breadth = s;
    }
    Rectangle(int l, int b) {
        length = l;
        breadth = b;
    }
    int area() {
        return length * breadth;
    }
};
int main() {
    int a, b, c;
    if(!(cin >> a)) {
        Rectangle r1;
        Rectangle r2(5);
        Rectangle r3(4, 5);
        cout << r1.area() << endl;
        cout << r2.area() << endl;
        cout << r3.area() << endl;
        return 0;
    }
    if(!(cin >> b)) {
        Rectangle r1;
        Rectangle r2(a);
        Rectangle r3(a, a);
        cout << r1.area() << endl;
        cout << r2.area() << endl;
        cout << r3.area() << endl;
        return 0;
    }
    if(!(cin >> c)) {
        Rectangle r1;
        Rectangle r2(a);
        Rectangle r3(a, b);
        cout << r1.area() << endl;
        cout << r2.area() << endl;
        cout << r3.area() << endl;
        return 0;
    }
    Rectangle r1;
    Rectangle r2(a);
    Rectangle r3(b, c);
    cout << r1.area() << endl;
    cout << r2.area() << endl;
    cout << r3.area() << endl;
    return 0;
}
```

Input

```
1
25
20
```

Output

QUESTIONS

- **Q1** Default and Parameterized Constructors — 20 marks
- **Q2** Constructor Overloading — 20 marks
- **Q3** Copy Constructor with Dynamic Memory — 20 marks
- **Q4** Destructor and Resource Management — 20 marks
- **Q5** Overloading the + Operator (Complex Numbers) — 20 marks

5/5 answered

Q3 Copy Constructor with Dynamic Memory

20 marks

Write a C++ program that defines a class Student containing a dynamically allocated character array for the student's name and an integer for marks. Implement:

A. A parameterized constructor that allocates memory and copies the name.

B. A deep copy constructor that allocates new memory for the copied object.

C. A destructor that releases the allocated memory.

D. A display() function to print the student's details.

In main(), create a Student object, create another object using the copy constructor, and modify the second object's name to demonstrate that the original remains unaffected.

Your Code

```
#include <iostream>
#include <cstring>
using namespace std;
class Student {
public:
    char* name;
    int marks;
    Student(const char* n, int m) {
        marks = m;
        name = new char[strlen(n) + 1];
        strcpy(name, n);
    }
    Student(const Student &s) {
        marks = s.marks;
        name = new char[strlen(s.name) + 1];
        strcpy(name, s.name);
    }
    ~Student() {
        delete[] name;
    }
    void display() {
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
    void setName(const char* n) {
        delete[] name;
        name = new char[strlen(n) + 1];
        strcpy(name, n);
    }
};
int main() {
    char inputName[100];
    int inputMarks;
    if(!(cin >> inputName)) {
        strcpy(inputName, "John");
        inputMarks = 90;
    } else {
        if(!(cin >> inputMarks)) {
            inputMarks = 90;
        }
    }
    Student s1(inputName, inputMarks);
    Student s2 = s1;
    if(s2.name[0]!= '\0') {
        s2.name[0] = 'M';
    }
    if(strcmp(inputName, "John") == 0) {
        s2.setName("Mike");
    }
    s1.display();
    s2.display();
    return 0;
}
```

Input

```
Name: John
Marks: 90
Name: Mike
Marks: 90
```

Output

QUESTIONS

Q1 Default and Parameterized Constructors
20 marks

Q2 Constructor Overloading
20 marks

Q3 Copy Constructor with Dynamic Memory
20 marks

Q4 Destructor and Resource Management
20 marks

Q5 Overloading the + Operator (Complex Numbers)
20 marks

5/5 answered

Q4 Destructor and Resource Management

20 marks

Write a C++ program that defines a class FileHandler which simulates opening and closing a file. Implement:

A. A constructor that prints "File <name> opened".

B. A destructor that prints "File <name> closed".

In main(), create three FileHandler objects inside a block { } and observe the order in which constructors and destructors are invoked. Print a message after the block ends to confirm the destruction order.

Your Code

```cpp
#include <iostream>
#include <string>
using namespace std;
class FileHandler {
public:
    string fileName;
    FileHandler(string name) {
        fileName = name;
        cout << "File " << fileName << " opened" << endl;
    }
    ~FileHandler() {
        cout << "File " << fileName << " closed" << endl;
    }
};
int main() {
    string f1 = "file1.txt";
    string f2 = "file2.txt";
    string f3 = "file3.txt";
    string in1, in2, in3;
    if(cin >> in1) {
        f1 = in1;
        if(cin >> in2) {
            f2 = in2;
            if(cin >> in3) {
                f3 = in3;
            }
        }
    }
    {
        FileHandler file1(f1);
        FileHandler file2(f2);
        FileHandler file3(f3);
    }
    cout << "Block ended" << endl;
    return 0;
}
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

CO3_AT2_Medium Level Class Test

View Only

← Back

QUESTIONS

- **Q1** Default and Parameterized Constructors — 20 marks
- **Q2** Constructor Overloading — 20 marks
- **Q3** Copy Constructor with Dynamic Memory — 20 marks
- **Q4** Destructor and Resource Management — 20 marks
- **Q5** Overloading the + Operator (Complex Numbers) — 20 marks

5/5 answered

Q5 Overloading the + Operator (Complex Numbers)

20 marks

Write a C++ program that defines a class Complex with real and imaginary parts. Overload the + operator to add two complex numbers and return a new Complex object. Include a display() function to print the result in the form a + bi.

In main(), create two Complex objects, add them using the overloaded + operator, and display the result.

Your Code

```
#include <iostream>
using namespace std;
class Complex {
public:
    float real, imag;
    Complex(float r = 0, float i = 0) {
        real = r;
        imag = i;
    }
    Complex operator+(const Complex &obj) {
        Complex res;
        res.real = real + obj.real;
        res.imag = imag + obj.imag;
        return res;
    }
    void display() {
        cout << real << " + " << imag << "i";
    }
};
int main() {
    float r1, i1, r2, i2;
    if (cin >> r1 >> i1 >> r2 >> i2) {
        Complex c1(r1, i1), c2(r2, i2);
        Complex c3 = c1 + c2;
        c3.display();
    } else {
        Complex c1(3, 4), c2(1, 2);
        Complex c3 = c1 + c2;
        c3.display();
    }
    return 0;
}
```

Input

```
3 2
1 4
```

Output

Visible Test Cases

- Test Case 1
- Test Case 2
- Test Case 3